TECHNICAL SELECTION GUIDE

Choosing the Right RAG Architecture

A scenario-based comparison of Vanilla, Graph, Vectorless, and other retrieval-augmented generation patterns — and how to match each to your data and your questions.

Prepared June 18, 2026 · Retrieval-Augmented Generation · Architecture Reference

01 How to read this

"Better" almost never means a single winner. The right RAG pattern is decided by two things: the **shape of your data** (a loose pile of documents vs. a few highly structured ones vs. clean records with a schema) and the **shape of your questions** (fuzzy/semantic vs. exact/analytical vs. relational). The table that follows maps each major pattern to the scenarios where it wins and the ones where it struggles. The decision guide on the final page collapses all of it into a few quick rules.

02 Comparison by scenario

RAG Type	How it works	Best-suited scenarios	Weak spots / avoid when
Vanilla RAG "Naive" baseline	Chunk documents, embed them, store in a vector DB, retrieve top-k by similarity, and stuff results into the prompt.	Large collections of loosely related documents; general Q&A and FAQ bots; "find me passages about X"; fastest pattern to stand up.	Precise multi-hop reasoning; exact aggregation; long structured docs where chunking severs context; queries where wording differs from meaning.
Graph RAG	Extract entities and relationships into a knowledge graph; retrieve via traversal plus community summaries (Microsoft GraphRAG style).	"How is A connected to B?"; multi-hop relationship questions; global/thematic synthesis across a corpus; connected domains like orgs, supply chains, research.	Expensive to build; quality bottlenecked by entity/relationship extraction; overkill for simple lookup; weak at exact counts.
Vectorless RAG e.g. PageIndex	Builds a hierarchical table-of-contents tree of a document; an LLM reasons over that structure to navigate to the right section — no embeddings, no vector DB, no nearest-neighbor search.	Long, structured long-form content: financial reports, legal contracts, regulatory filings, policy docs, academic papers; when logical organization matters more than surface similarity; need for a traceable retrieval path.	Large collections of unrelated or loosely structured documents (vector search wins here); adds per-query LLM reasoning latency and cost; depends on documents actually having structure.
Hybrid RAG	Combines vector (semantic) and keyword/BM25 (lexical) retrieval, usually with a reranking stage.	Mixed queries where exact terms — codes, IDs, names, acronyms — and semantic meaning both matter; enterprise and technical search.	More moving parts to tune; fusion weighting needs care; still inherits the limits of chunking.
Agentic RAG	An agent plans, selects tools and sources, issues multiple retrievals, validates results, and iterates.	Complex multi-step questions; routing across heterogeneous sources (vector + SQL + graph + web); when one retrieval pass isn't enough.	Latency and cost from many LLM calls; harder to debug; can loop or over-retrieve without guardrails.
Text-to-Query RAG SQL / Cypher	An LLM translates the question into SQL or Cypher that runs against a database or graph.	Structured, clean data with a known schema; exact aggregations, filters, rankings ("how many," "sum," "top 5"); auditable answers.	Requires correct query generation; fragile on complex or ambiguous schemas without grounding; not for unstructured text.
HyDE / Query Transform	The LLM generates a hypothetical answer (or rewrites the query), then embeds that for retrieval.	Short or vague queries; vocabulary-mismatch problems where the question doesn't lexically resemble the source.	Adds a generation step; can drift if the hypothetical answer is wrong; still vector-based underneath.
Self-RAG / CRAG	The model critiques retrieved context, decides whether to retrieve again, discards bad results, or falls back to web search.	High-stakes accuracy needs; reducing hallucination; corpora with uneven or noisy quality.	Extra LLM passes add cost and latency; more complex pipeline to operate.

RAG Type	How it works	Best-suited scenarios	Weak spots / avoid when
Multimodal RAG	Retrieves across text plus images, tables, and charts using vision embeddings or vision-native indexing.	Documents where answers live in figures, tables, diagrams, or scanned pages; technical manuals and slide decks.	Heavier infrastructure; embedding and parsing quality for non-text content is still maturing.

03 The quick decision guide

If you only remember one page, make it this one. Match your situation on the left to the recommended starting point on the right.

A loose pile of many documents, semantic lookup	**Vanilla → Hybrid if exact terms matter**
Relationships and connections across entities	**Graph RAG**
A few long, well-structured documents needing precise, traceable answers	**Vectorless**
Clean structured data, exact numbers and aggregations	**Text-to-Query**
Complex, multi-source, multi-step questions	**Agentic (wrapping the others)**
Answers live in figures, tables, or scanned pages	**Multimodal**

> **These aren't mutually exclusive.** Production systems frequently combine them — for example, an agentic router that sends precise questions to text-to-Cypher, structural questions to a Vectorless index, and fuzzy semantic ones to a hybrid vector store over the same underlying data. Start with the simplest pattern that fits your dominant question type, then add routing only when a second question type proves it's needed.

Vectorless RAG / PageIndex characteristics reflect the framework published by VectifyAI (Sep 2025) and current public documentation as of June 2026. All guidance is directional; validate against your own corpus and evaluation set.